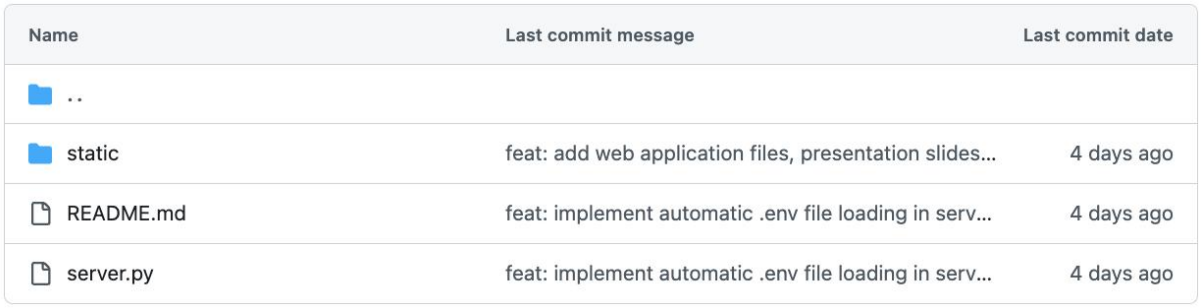
Rel Guzman

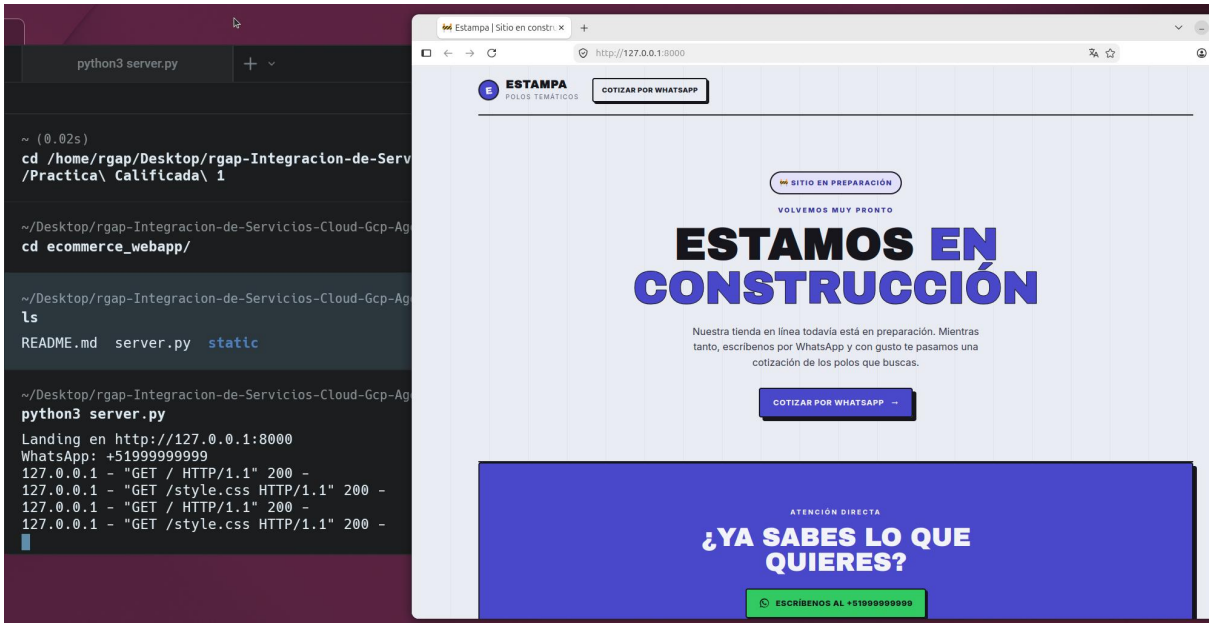
Prompt de consulta a la IA:
Resolvamos esta practica parte por parte, cada parte tiene sus pasos dentro y de cada parte se saca unas evidencias.
Pero recorre detalladamente cada paso sin salirte del tema general.
Menciona cada enunciado literal y resuelve.
Primero la parte 1. No agregues tanto detalle. el objetivo es las evidencias.

Parte 1 — Recibir el servidor e inventariar el entorno — 1 punto

1.1. Describe brevemente qué entrega desarrollo y qué debes resolver como sysadmin. Diferencia visitante, desarrollador y administrador.

El equipo de desarrollo me entregó una aplicación web **ecommerce_webapp**

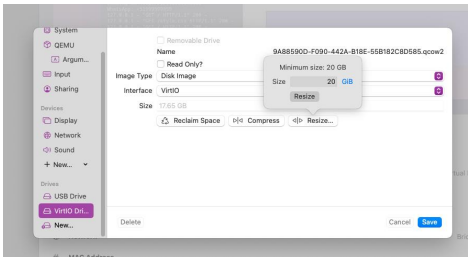


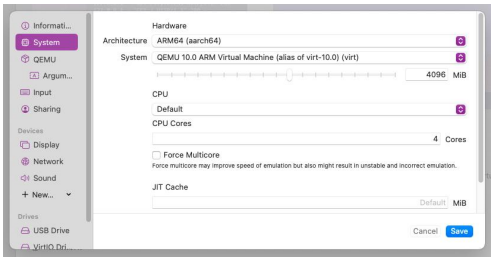


Roles relacionados al sistema

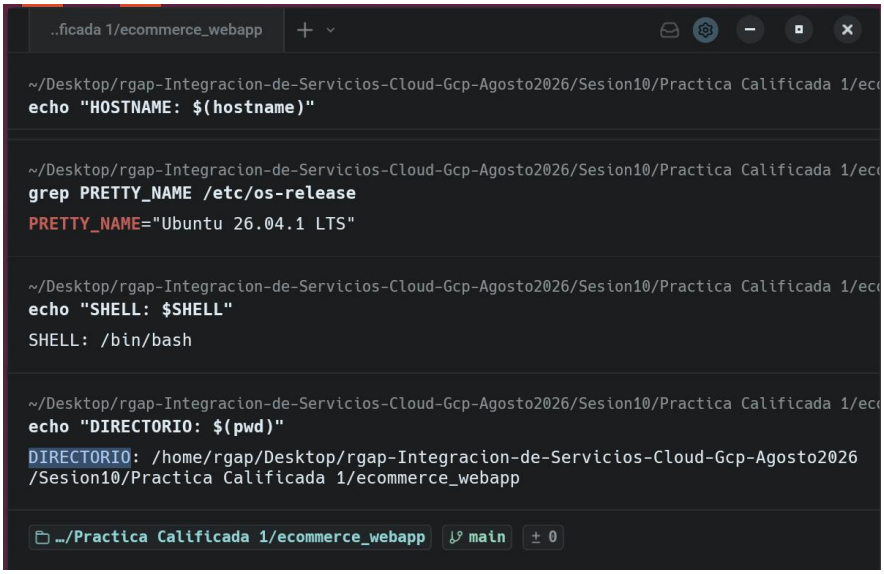
- **Visitante:** Accede al navegador, entra a la webapp y usa el enlace para cotizar por whatsapp.
- **Desarrollador:** Entrega el código funcional de la webapp.
- **Administrador del sistema (sysadmin):** Recibe el software y prepara el sistema donde se ejecutará: Ubuntu, paquetes, red, usuarios, permisos, systemd, firewall, Nginx, procesos, puertos y comprobaciones.

1.2. Configura o reutiliza una VM Ubuntu. Registra hipervisor, tipo, versión de Ubuntu, CPU, RAM y disco asignados; justifica que los recursos caben en el equipo físico.



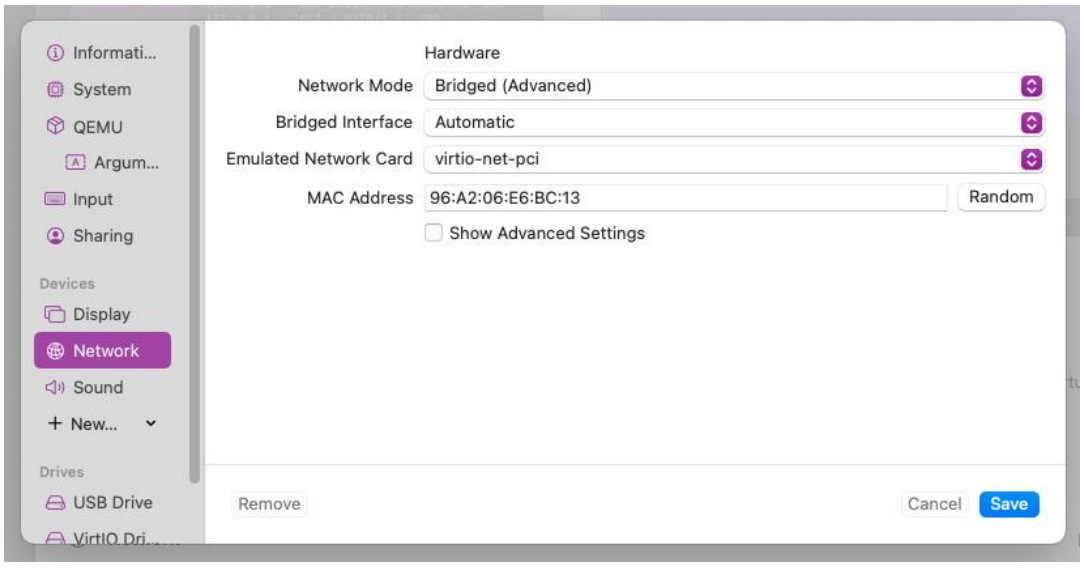


1.3. Configura el adaptador en modo puente sobre la interfaz física activa.
1.4. Identifica usuario, hostname, sistema operativo, shell y directorio de trabajo. Utiliza los comandos de las sesiones previas.

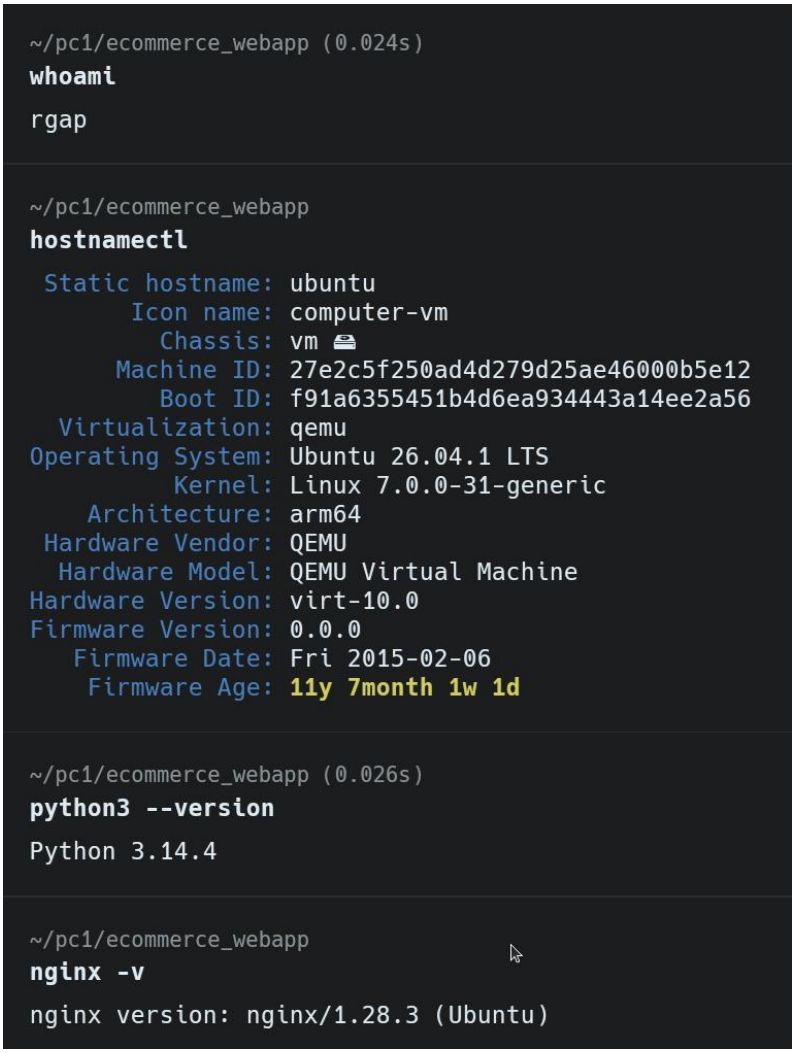


1.5. Crea un directorio de trabajo y copia la aplicación completa. Revisa su README, estructura y variables HOST, PORT, WHATSAPP_NUMBER y WHATSAPP_MESSAGE.
1.6. Actualiza el índice de paquetes e instala las herramientas necesarias: Python 3, Nginx, UFW, cliente HTTP y herramientas de observación de conexiones. El servidor recibido usa la biblioteca estándar de Python.

Evidencia E01: configuración de la VM con recursos y modo puente visibles.



Evidencia E02: identificación de Ubuntu y usuario, estructura de ecommerce_webapp y versiones de Python y Nginx.



Parte 2 — Reconocer la red y comprobar la salida a Internet — 2 puntos

2.1. Obtén VM_IP, prefijo de red, interfaz activa y ROUTER_IP desde Ubuntu, mediante ip addr e ip route o comandos equivalentes.
2.2 Obtén HOST_IP desde el equipo físico. Comprueba con el prefijo que cliente y VM pertenecen a la misma subred.
2.3. Registra los siguientes datos con el comando o pantalla que sustenta cada uno.

HOST_IP =
VM_IP =
PREFIJO_RED =
INTERFAZ_VM =
ROUTER_IP =
MODE = Bridged
DEST_NAME = www.google.com
DEST_IP =
PROTOCOL =
PORT =

2.4. Reproduce el análisis de tarea: resuelve el nombre de destino y realiza desde la VM una conexión HTTPS sobre TCP, por ejemplo con curl -I https://www.google.com. Observa el destino efectivo de esa conexión y su puerto; no copies la IP del ejemplo de clase.
2.5. Explica dónde intervienen DNS y el NAT de salida del router. No confundas el puente del hipervisor con el NAT del router. Puedes representar la dirección WAN como IP pública, sin registrar su valor.
2.6. Diferencia esa salida a Internet del acceso local PC_HOST → VM: en la misma subred no se necesita NAT ni una ruta por Internet para consultar la landing. Un ping no demuestra por sí solo que HTTP funcione.

E03: direcciones, prefijo, interfaz y ruta de la VM, junto con la IP del host.

- HOST_IP =
- VM_IP = 192.168.100.158
- PREFIJO_RED = /24
- INTERFAZ_VM = enp0s1
- ROUTER_IP = 192.168.100.1
- MODE = Bridged
- DEST_NAME = www.google.com
- DEST_IP =
- PROTOCOL =
- PORT =

```
~/pc1/ecommerce_webapp (0.028s)
ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s1           UP             192.168.100.158/24 fe80::94a2:6ff:fee6:bc13/64

~/pc1/ecommerce_webapp
ip route
default via 192.168.100.1 dev enp0s1 proto dhcp src 192.168.100.158 metric 100
192.168.100.0/24 dev enp0s1 proto kernel scope link src 192.168.100.158 metric 10
0
```

E04: resolución de nombre y evidencia de la conexión HTTPS al destino real por TCP/443, mediante captura de tráfico o herramienta equivalente.

- HOST_IP = 192.168.100.170
- VM_IP = 192.168.100.158
- PREFIJO_RED = /24
- INTERFAZ_VM = enp0s1
- ROUTER_IP = 192.168.100.1
- MODE = Bridged
- DEST_NAME = www.google.com
- DEST_IP = 142.251.0.147
- PROTOCOL = TCP / HTTPS
- PORT = 443

```
~/pc1/ecommerce_webapp (2.846s)
ping www.google.com
PING www.google.com (142.251.0.147) 56(84) bytes of
64 bytes from cj-in-f147.1e100.net (142.251.0.147):
ms
64 bytes from cj-in-f147.1e100.net (142.251.0.147):
ms
64 bytes from cj-in-f147.1e100.net (142.251.0.147):
ms
```

Parte 3 — Reproducir los diagramas de comunicación TCP — 3 puntos

3.1. En archivos de laboratorio separados de la webapp, adapta los ejemplos de la sesión 07 para crear un servidor TCP de saludo y un cliente TCP.
3.2. En la VM, el servidor debe quedar a la escucha en 0.0.0.0:8000 y aceptar la conexión entrante del cliente.
3.3. En PC_HOST, el cliente debe conectarse a VM_IP:8000. Configura temporalmente el acceso necesario en UFW para ese origen y puerto.
3.4. Envía Hola desde el cliente; el servidor debe recibirlo y responder Hola PC; el cliente debe mostrar esa respuesta. Ambos mensajes deben viajar sobre la misma conexión aceptada.
3.5. Muestra las direcciones y puertos reales de ambos extremos mediante el programa o herramientas del sistema. Registra el puerto del cliente y el puerto de escucha del servidor.
3.6. Captura el tráfico de la prueba y guarda evidencia de la comunicación entre el cliente y el servidor.
Detén este servidor y comprueba que liberó el puerto 8000 antes de iniciar la webapp.

E05: terminal del servidor y terminal del cliente con los dos mensajes, las IP y los puertos reales.

```
~/pc1/tcp_lab (2m 26.00s)
python3 servidor_tcp.py

Servidor escuchando en 0.0.0.0:8000
Conexión aceptada desde 192.168.100.170:57912
Extremo servidor: ('192.168.100.158', 8000)
Recibido: Hola
Enviado: Hola PC
Presiona ENTER para cerrar la conexión...
```

```
~/Desktop » python cliente_tcp.py
Cliente local: ('192.168.100.170', 57912)
Servidor remoto: ('192.168.100.158', 8000)
Enviado: Hola
Recibido: Hola PC
Presiona ENTER para cerrar la conexión...
```

E06: evidencia del servicio en escucha y de la comunicación entre el cliente y el servidor. Puedes mantener brevemente la conexión abierta para observarla con ss.

```
~/pc1/tcp_lab (0.032s)
ss -tnp | grep :8000

ESTAB 0      0      192.168.100.158:8000  192.168.100.170:57849 users:(("pytho
n3",pid=20529,fd=4))

~/pc1/tcp_lab
```

```
....*B.:
21:31:25.659762 IP 192.168.100.158.8000 > 192.168.100.170.57912: Flags [P.], s
eq 1:8, ack 5, win 128, options [nop,nop,TS val 781781142 ecr 709015866], leng
th 7
E..;.
@.@.n...d...d..@.8.X...!.:....%.....
....*B.:Hola PC
21:31:25.660220 IP 192.168.100.170.57912 > 192.168.100.158.8000: Flags [.], ac
k 8, win 2059, options [nop,nop,TS val 709015868 ecr 781781142], length 0
E..4..@.@..*..d...d..8.@.!.:.X......6?.....
*B.<....
21:31:26.765281 IP 192.168.100.170.57849 > 192.168.100.158.8000: Flags [F.], s
eq 0, ack 1, win 2059, options [nop,nop,TS val 1745860205 ecr 38324073], lengt
h 0
E..4..@.@..*..d...d....@wv.=.I......".....
h..m.H.i
```

Parte 4 — Desplegar la landing directamente en Python — 3 puntos

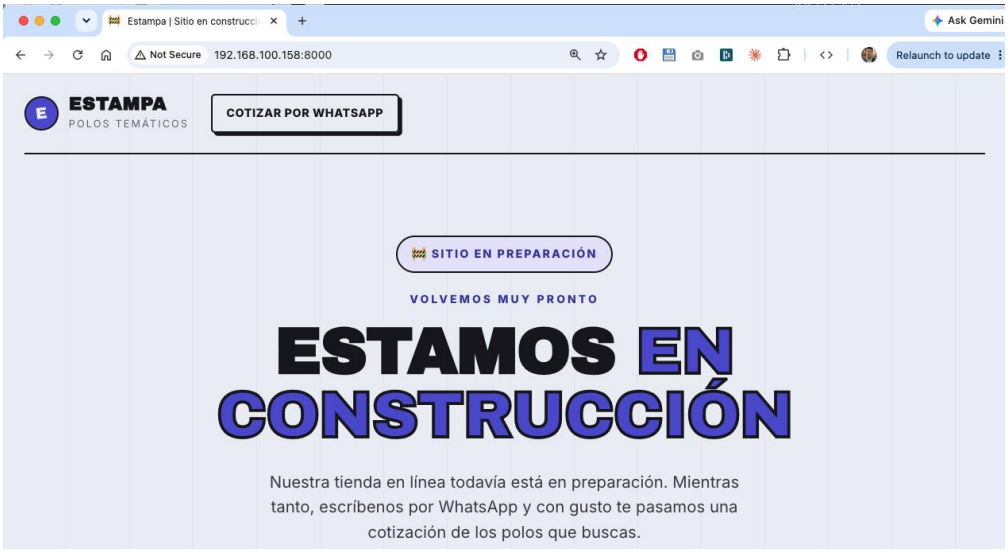
4.1. Inicia el proyecto entregado con Python 3 y las variables HOST=0.0.0.0 y PORT=8000. Usa las variables de WhatsApp con valores de prueba identificados como tales. Coloca la configuración en un archivo .env junto a server.py: el servidor lo carga automáticamente al arrancar (las variables ya definidas en el entorno tienen prioridad).

```
~/pc1/ecommerce_webapp (3m 58.00s)
python3 server.py

Landing en http://0.0.0.0:8000
WhatsApp: +51999999999
192.168.100.170 - "GET / HTTP/1.1" 200 -
192.168.100.170 - "GET /style.css HTTP/1.1" 200 -
```

4.2. Comprueba desde la VM qué proceso escucha en 8000 y en qué dirección.

4.3. Desde el navegador de PC_HOST, abre http://VM_IP:8000/. Debe verse Estampa, el anuncio de construcción, los estilos y los enlaces de cotización.



4.4. Desde el cliente, solicita /, /index.html, /style.css y /productos. Registra los códigos HTTP y el tipo de contenido. Para /productos el resultado esperado es 404.

```
[~ » curl -I http://192.168.100.158:8000/index.html
HTTP/1.0 200 OK
Server: LandingEnConstruccion/1.0 Python/3.14.4
Date: Tue, 15 Sep 2026 04:47:28 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 4554

[~ » curl -I http://192.168.100.158:8000/productos
HTTP/1.0 404 Not Found
Server: LandingEnConstruccion/1.0 Python/3.14.4
Date: Tue, 15 Sep 2026 04:47:46 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 19
```

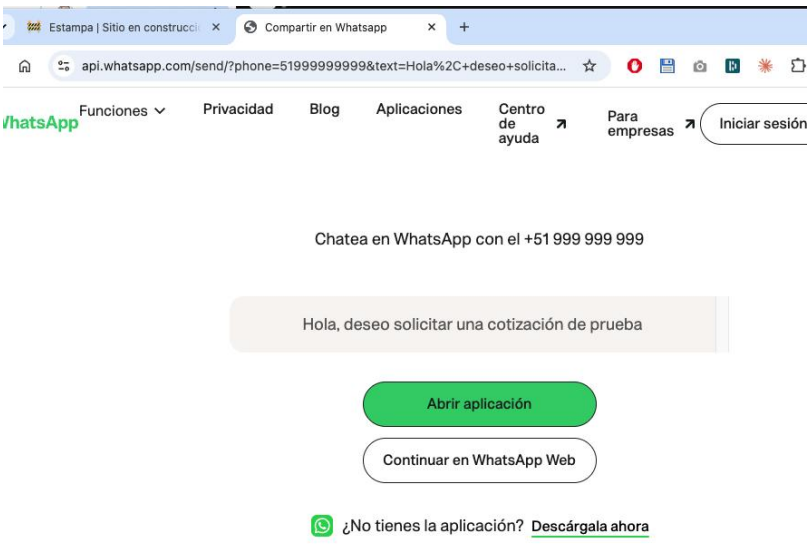
4.5. Adapta el cliente HTTP de la sesión 07 para solicitar / a VM_IP:8000 sobre una conexión TCP. Envía una petición HTTP completa, con versión, cabecera Host y terminación de cabeceras; lee la respuesta completa y muestra estado, cabeceras y una porción del HTML.

```
~/Desktop » python cliente_http.py
Conectado a: ('192.168.100.158', 8000)
Cliente: ('192.168.100.170', 59570)

=== ESTADO Y CABECERAS ===
HTTP/1.0 200 OK
Server: LandingEnConstruccion/1.0 Python/3.14.4
Date: Tue, 15 Sep 2026 04:53:32 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 4554

=== FRAGMENTO HTML ===
<!doctype html>
<html lang="es">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <meta name="theme-color" content="#5a44d6" />
    <title>Estampa | Sitio en construcción</title>
    <meta
      name="description"
      content="Estamos preparando nuestra tienda de polos temáticos. Mientras
p."
    />
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https
```

4.6. Inspecciona el enlace generado de WhatsApp: verifica número y mensaje codificado. Basta inspeccionar el destino; no envíes mensajes.

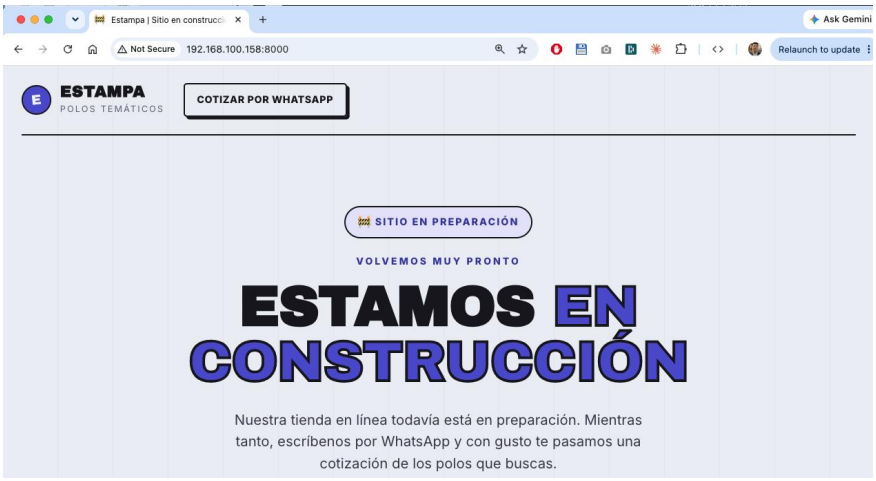


4.7. Explica por qué 0.0.0.0 es una dirección de escucha y el navegador debe usar la IP real de la VM.

0.0.0.0 indica que Python escucha conexiones en todas las interfaces de red disponibles de la VM.

Por ello, desde PC_HOST el navegador debe conectarse utilizando la dirección real de la VM, VM_IP:8000.

E07: landing desde el host, con la URL http://VM_IP:8000/ visible, y proceso de escucha en la VM.



E08: respuestas HTTP de las cuatro rutas y respuesta recibida por el cliente HTTP de la sesión 07, incluyendo un fragmento reconocible de la landing.

```
~ » curl -I http://192.168.100.158:8000/index.html
HTTP/1.0 200 OK
Server: LandingEnConstruccion/1.0 Python/3.14.4
Date: Tue, 15 Sep 2026 04:47:28 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 4554

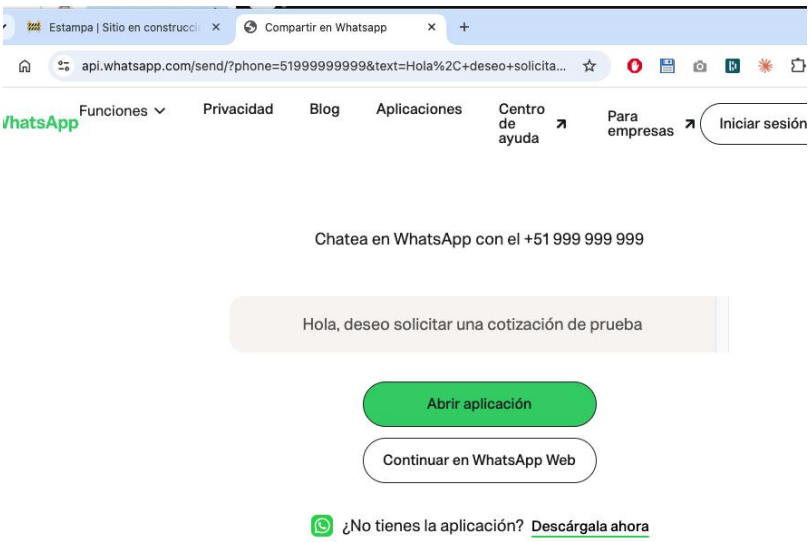
~ » curl -I http://192.168.100.158:8000/productos
HTTP/1.0 404 Not Found
Server: LandingEnConstruccion/1.0 Python/3.14.4
Date: Tue, 15 Sep 2026 04:47:46 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 19
```

```
~/Desktop » python cliente_http.py
Conectado a: ('192.168.100.158', 8000)
Cliente: ('192.168.100.170', 59570)

=== ESTADO Y CABECERAS ===
HTTP/1.0 200 OK
Server: LandingEnConstruccion/1.0 Python/3.14.4
Date: Tue, 15 Sep 2026 04:53:32 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 4554

=== FRAGMENTO HTML ===
<!doctype html>
<html lang="es">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <meta name="theme-color" content="#5a44d6" />
    <title>Estampa | Sitio en construcción</title>
    <meta
      name="description"
      content="Estamos preparando nuestra tienda de polos temáticos. Mientras
p."
    />
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https
```

E09: enlace de cotización generado con la configuración de prueba, sin marcadores {{WHATSAPP_LINK}} pendientes de sustituir.



Parte 5 — Administrar usuarios, permisos y servicio — 2 puntos

5.1. Define una cuenta sin privilegios administrativos para ejecutar la aplicación. Documenta propietario, grupo y permisos de los archivos y directorios necesarios.

5.2. Mantén el archivo .env con las variables HOST, PORT, WHATSAPP_NUMBER y WHATSAPP_MESSAGE en el directorio de la aplicación, junto a server.py (el mismo que usaste en la parte 4). Es la única fuente de configuración del servicio: server.py lo carga al arrancar.

5.3. Aplica permisos mínimos al código y al archivo .env. No uses chmod 777.

5.4. Restringe el .env a las cuentas que necesiten leerlo (por ejemplo propietario administrativo y grupo del servicio, sin acceso para el resto). La cuenta del servicio debe poder leer el código, el .env y los recursos estáticos, y recorrer sus directorios. No necesita permisos administrativos para escuchar en 8000.

5.5. Detén la ejecución manual y configura una unidad systemd para la aplicación, con usuario, directorio de trabajo y comando de inicio que invoque directamente Python 3 con la ruta de server.py. server.py carga el .env que está junto a él, así que no necesitas declarar variables en la unidad. El nombre del servicio debe quedar registrado en el informe.

5.6. Cambia en el .env el backend a HOST=127.0.0.1 y PORT=8000, como requiere la arquitectura final, y reinicia el servicio.

5.7. Demuestra inicio, consulta de estado, parada, reinicio y habilitación al arranque. Diferencia start de enable, y restart de reload; no presupongas que tu unidad Python soporta recarga.

5.8. Crea un alias o función persistente para consultar el estado del servicio y demuéstralo desde una nueva sesión de shell.

```
~ (25.259s)
nano ~/.bashrc

~ (0.053s)
source ~/.bashrc

~ (0.052s)
estado-estampa
● estampa.service - Landing Estampa
     Loaded: loaded (/etc/systemd/system/estampa.service; enabled; preset: ena
bled)
     Active: active (running) since Tue 2026-09-15 09:24:49 -05; 4min 25s ago
 Invocation: 0efda651e2cc46d7a41c7605a94fe96c
   Main PID: 80149 (python3)
      Tasks: 1 (limit: 2263)
     Memory: 11.2M (peak: 11.4M)
        CPU: 77ms
     CGroup: /system.slice/estampa.service
             └─80149 /usr/bin/python3 /opt/estampa/ecommerce_webapp/server.py

Sep 15 09:24:49 ubuntu systemd[1]: Started estampa.service - Landing Estampa.
Hint: Some lines were ellipsized, use -l to show in full.
```

E10: propietario y permisos del código y del archivo .env, y usuario efectivo del proceso Python.

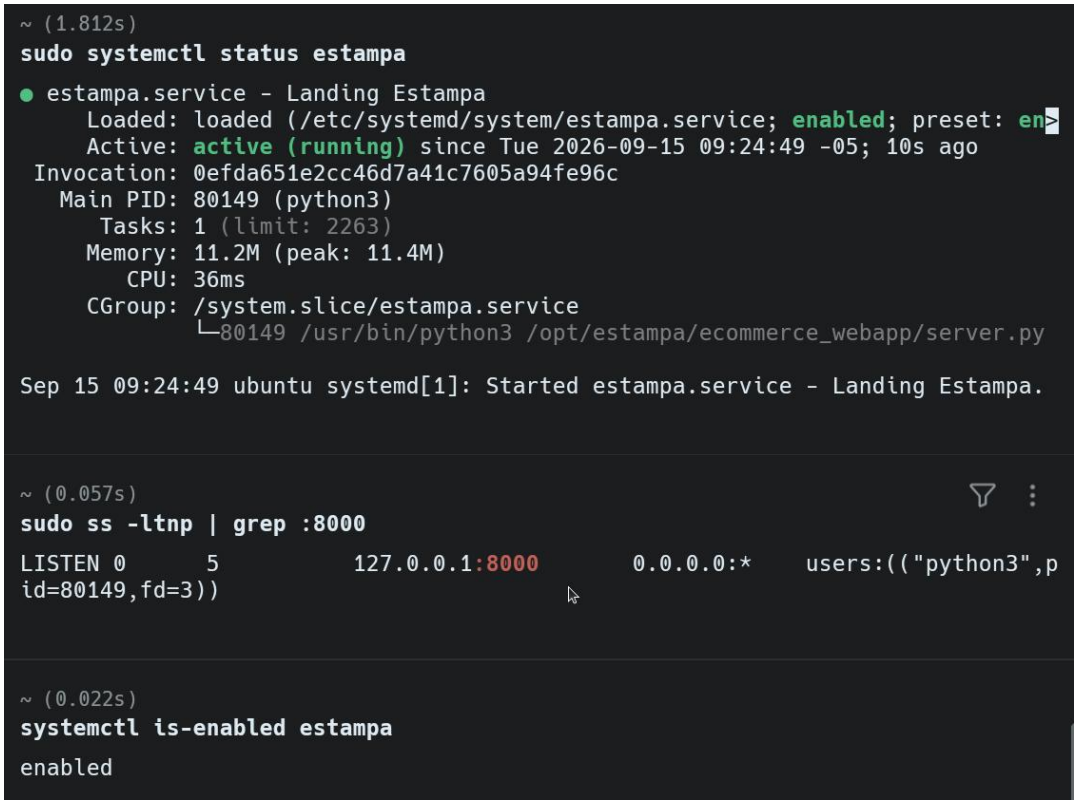
```
/opt (0.033s)
sudo ls -la /opt/estampa/ecommerce_webapp
total 28
drwxr-x--- 3 root estampa 4096 Sep 15 00:30 .
drwxr-x--- 3 root estampa 4096 Sep 15 00:16 ..
-rw-r----- 1 root estampa  118 Sep 15 00:30 .env
-rw-r----- 1 root estampa 2184 Sep 15 00:17 README.md
-rw-r----- 1 root estampa 4103 Sep 15 00:17 server.py
drwxr-x--- 2 root estampa 4096 Sep 14 20:10 static
```

```
/opt (2m 20.92s)
sudo -u estampa python3 /opt/estampa/ecommerce_webapp/server.py
Landing en http://127.0.0.1:8000
WhatsApp: +51999999999
^C
Deteniendo servidor...

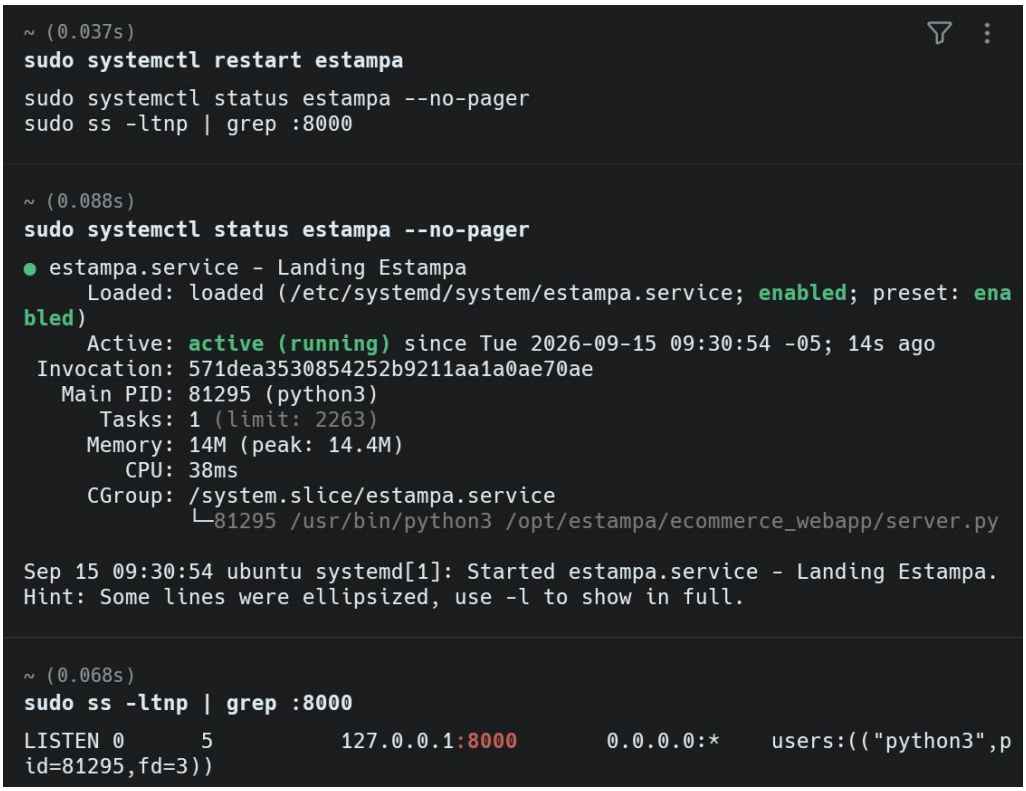
/opt (0.045s)
sudo -u rgap python3 /opt/estampa/ecommerce_webapp/server.py
python3: can't open file '/opt/estampa/ecommerce_webapp/server.py': [Errno 13]
 Permission denied

/opt (7.669s)
sudo -u root python3 /opt/estampa/ecommerce_webapp/server.py
Landing en http://127.0.0.1:8000
WhatsApp: +51999999999
^C
Deteniendo servidor...
```

E11: contenido de la unidad systemd, su habilitación al arranque (enable) y el alias o función de estado funcionando desde una nueva sesión de shell. No muestres contraseñas ni claves privadas.



E12: .env con el backend cambiado a HOST=127.0.0.1 y PORT=8000, servicio reiniciado, y demostración del ciclo start / status / stop / restart, con la explicación de la diferencia entre start y enable, y entre restart y reload.



Parte 6 — Publicar mediante Nginx — 3 puntos

6.1. Adapta la configuración de proxy inverso de la sesión 09. Nginx debe escuchar en el puerto 80 y reenviar las rutas del sitio hacia http://127.0.0.1:8000.

6.2. Comprueba la configuración con nginx -t antes de aplicarla y recarga Nginx. Verifica que el bloque activo atienda el acceso por IP y no muestre la página predeterminada de Nginx.

6.3. Desde la VM, verifica que el backend responde en http://127.0.0.1:8000/.

6.4. Desde PC_HOST, visita http://VM_IP/ y repite las pruebas de rutas de la parte 4 a través de Nginx. Comprueba HTML, CSS y enlaces de WhatsApp.

6.5. Muestra los listeners: Nginx en 0.0.0.0:80 y Python en 127.0.0.1:8000. Python ya no debe escuchar en 0.0.0.0:8000 ni en una dirección externa equivalente.

6.6. Observa una solicitud en la interfaz de red de la VM y en loopback. Identifica los dos tramos TCP: HOST_IP:puerto_efimero → VM_IP:80 y 127.0.0.1:otro_puerto_efimero → 127.0.0.1:8000.

6.7. Relaciona la petición y respuesta con los registros de Nginx y del backend. Explica que Nginx establece una conexión al backend: no es una sola conexión TCP que atraviesa ambos procesos.

```
192.168.100.170 - - [15/Sep/2026:10:09:03 -0500] "GET /style.css HTTP/1.1" 200 35521 "
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36"
```

```
sudo journalctl -u estampa -f
[sudo: authenticate] Password:
Sep 15 09:24:38 ubuntu systemd[1]: Stopped estampa.service - Landing Estampa.
Sep 15 09:24:38 ubuntu systemd[1]: Started estampa.service - Landing Estampa.
```

E13: configuración activa del proxy, validación satisfactoria de Nginx y listeners finales.

```
~ (0.03s)
cat /etc/nginx/sites-available/estampa
server {
    listen 80;
    server_name _;

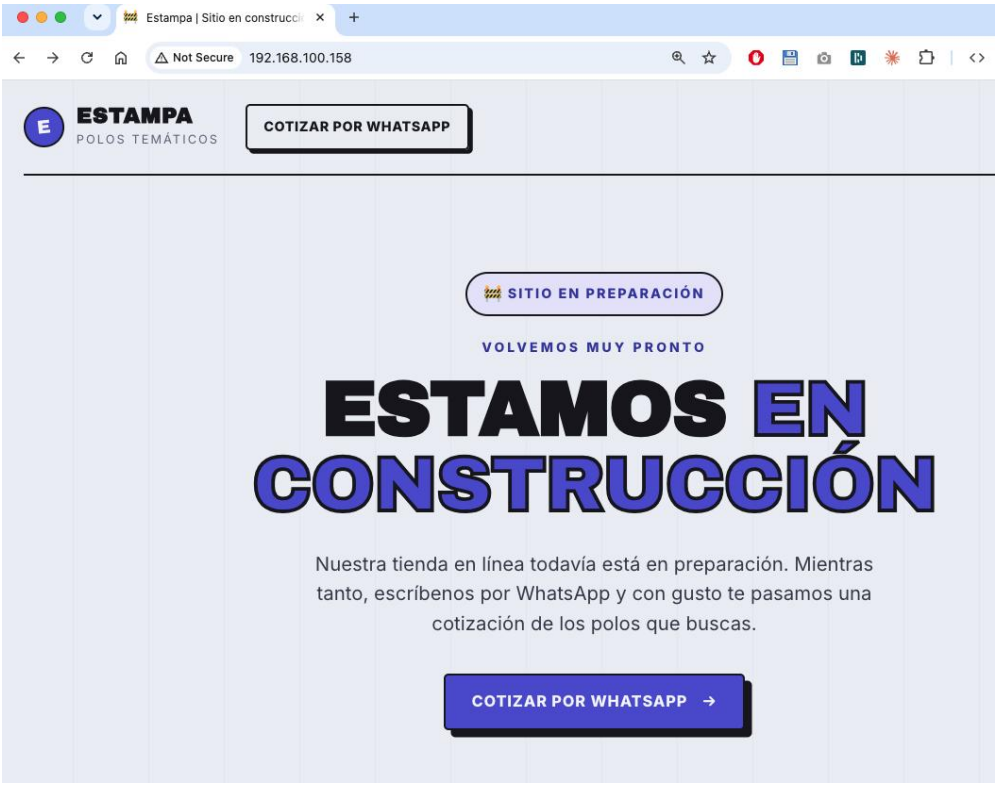
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

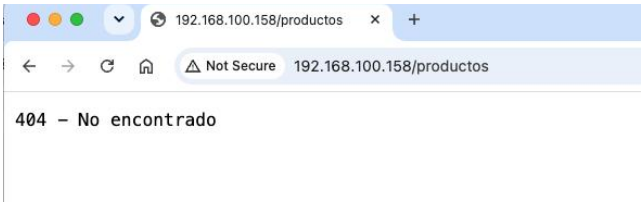
```
~
sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

```
~ (0.053s)
sudo ss -ltnp | grep -E ':80|:8000'
LISTEN 0      5          127.0.0.1:8000       0.0.0.0:*    users:(("python3",p
id=81295,fd=3))

LISTEN 0      511          0.0.0.0:80         0.0.0.0:*    users:(("nginx",pid
=83405,fd=5),("nginx",pid=83403,fd=5),("nginx",pid=83401,fd=5),("nginx",pid=83
400,fd=5),("nginx",pid=13402,fd=5))
```

E14: landing final en http://VM_IP/, respuestas de las cuatro rutas y registros de ambos servicios correspondientes a las pruebas.





E15: tráfico de los dos tramos, con extremos, petición y respuesta identificados. Presenta varios recortes legibles si una sola imagen no permite verlos.

Tramo 1

```
10:01:43.042330 enp0s1 In  IP 192.168.100.170.65339 > 192.168.100.158.80: Flags [S], seq 3990756806, win 65535, option
```

Tramo 2

```
10:01:43.044018 lo    In  IP 127.0.0.1.56808 > 127.0.0.1.8000: Flags [S], seq 4154130970, win 65495, options [mss 6549
```

Parte 7 — Controlar el acceso y recuperar una falla — 2 puntos

7.1. Deja UFW activo: permite el acceso HTTP de la red de evaluación; retira el permiso temporal de 8000 y registra las reglas finales.

7.2. Desde el host, comprueba que http://VM_IP/ funciona y que http://VM_IP:8000/ no es accesible. Desde la VM, confirma que http://127.0.0.1:8000/ sí responde. Usa tiempos de espera limitados para las pruebas negativas.

7.3. Explica por separado la restricción por dirección de escucha y la política del firewall: una prueba fallida por sí sola no identifica cuál de las dos impide la conexión.

Python escucha únicamente en 127.0.0.1:8000, por lo que no acepta conexiones externas directamente.
Además, UFW no permite acceso externo al puerto 8000. Son dos controles diferentes.

7.4. Reproduce el bloqueo por IP de la sesión 08, limitado al tráfico HTTP del host. Coloca la regla antes de la autorización correspondiente, abre una conexión nueva y demuestra el fallo. Elimina esa regla temporal y demuestra la recuperación. Mantén disponible la consola de la VM.

7.5. Simula una incidencia deteniendo únicamente el servicio Python. Solicita de nuevo la landing a través de Nginx, registra el error observado —normalmente 502 Bad Gateway— y diagnostica usando estado del servicio, puertos en escucha y logs.




```
~ (0.055s)
sudo journalctl -u estampa -n 20
Sep 15 09:24:38 ubuntu systemd[1]: Stopped estampa.service - Landing Estampa.
Sep 15 09:24:38 ubuntu systemd[1]: Started estampa.service - Landing Estampa.
Sep 15 09:24:49 ubuntu systemd[1]: Stopping estampa.service - Landing Estampa...
Sep 15 09:24:49 ubuntu systemd[1]: estampa.service: Deactivated successfully.
Sep 15 09:24:49 ubuntu systemd[1]: Stopped estampa.service - Landing Estampa.
Sep 15 09:24:49 ubuntu systemd[1]: Started estampa.service - Landing Estampa.
Sep 15 09:30:54 ubuntu systemd[1]: Stopping estampa.service - Landing Estampa...
Sep 15 09:30:54 ubuntu systemd[1]: estampa.service: Deactivated successfully.
Sep 15 09:30:54 ubuntu systemd[1]: Stopped estampa.service - Landing Estampa.
Sep 15 09:30:54 ubuntu systemd[1]: Started estampa.service - Landing Estampa.
Sep 15 10:31:35 ubuntu systemd[1]: Stopping estampa.service - Landing Estampa...
Sep 15 10:31:35 ubuntu systemd[1]: estampa.service: Deactivated successfully.
Sep 15 10:31:35 ubuntu systemd[1]: Stopped estampa.service - Landing Estampa.
Sep 15 10:32:39 ubuntu systemd[1]: Started estampa.service - Landing Estampa.
Sep 15 10:32:50 ubuntu systemd[1]: Stopping estampa.service - Landing Estampa...
Sep 15 10:32:50 ubuntu systemd[1]: estampa.service: Deactivated successfully.
Sep 15 10:32:50 ubuntu systemd[1]: Stopped estampa.service - Landing Estampa.

~ (0.034s)
sudo tail -n 20 /var/log/nginx/error.log
2026/09/15 10:32:20 [error] 87530#87530: *63 connect() failed (111: Connection refused) whil
_, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:8000/", host: "192.168.100.158"
2026/09/15 10:32:52 [error] 87530#87530: *63 connect() failed (111: Connection refused) whil
_, request: "GET / HTTP/1.1", upstream: "http://127.0.0.1:8000/", host: "192.168.100.158"
```

7.6. Recupera el servicio y comprueba una nueva respuesta 200. Explica por qué Nginx activo no garantiza que el backend funcione.

7.7. Reinicia la VM y verifica que Nginx y la aplicación arrancan automáticamente y la landing vuelve a responder. Si cambia la IP por DHCP, actualiza la ficha y los diagramas finales.

E16: UFW activo, reglas finales y pruebas comparadas de acceso al puerto 80, rechazo o falta de acceso externo a 8000, y respuesta local del backend.

```
~ (0.082s)
sudo ufw status numbered
Status: active

     To                         Action      From
     --                         ------      ----
[ 1] 80/tcp                     ALLOW IN    Anywhere
[ 2] 80/tcp (v6)                ALLOW IN    Anywhere (v6)
```

E17: bloqueo HTTP temporal por origen, fallo desde el cliente, eliminación de la regla y recuperación.

```
~ (0.082s)
sudo ufw status numbered

Status: active

     To                         Action      From
     --                         ------      ----
[ 1] 80/tcp                     ALLOW IN    Anywhere
[ 2] 80/tcp (v6)                ALLOW IN    Anywhere (v6)
```

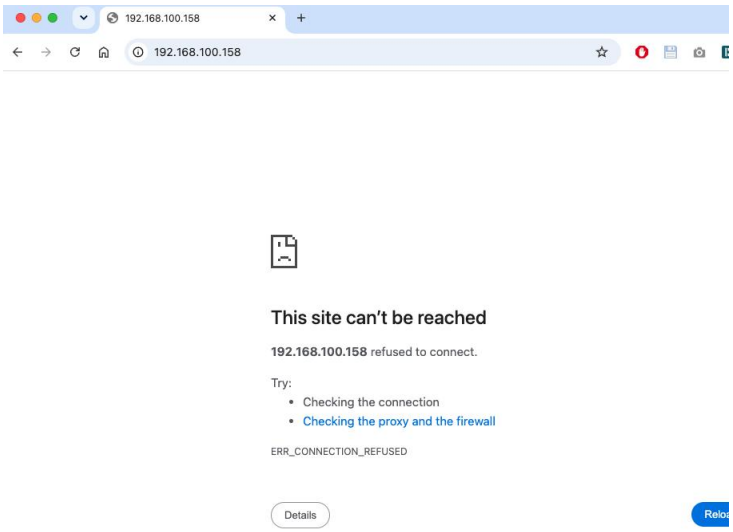
```
~ (0.133s)
sudo ufw insert 1 deny from 192.168.100.170 to any port 80 proto tcp

Rule inserted
```

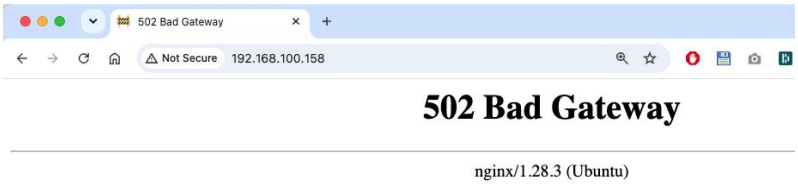
```
~ (0.087s)
sudo ufw status numbered

Status: active

     To                         Action      From
     --                         ------      ----
[ 1] 80/tcp                     DENY IN     192.168.100.170
[ 2] 80/tcp                     ALLOW IN    Anywhere
[ 3] 80/tcp (v6)                ALLOW IN    Anywhere (v6)
```



E18: incidencia del backend, diagnóstico, recuperación y verificación de ambos servicios después del reinicio de la VM.




```
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enable
     Active: active (running) since Tue 2026-09-15 10:38:58 -05; 2
 Invocation: c4f9eb3f19b94772a7634f834a3fe522
       Docs: man:nginx(8)
    Process: 1297 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on;
    Process: 1304 ExecStart=/usr/sbin/nginx -g daemon on; master_p
   Main PID: 1305 (nginx)
      Tasks: 5 (limit: 2263)
     Memory: 5.8M (peak: 8.3M, swap: 2.4M, swap peak: 2.4M)
        CPU: 22ms
     CGroup: /system.slice/nginx.service
             ├─1305 "nginx: master process /usr/sbin/nginx -g daem
             ├─1306 "nginx: worker process"
             ├─1307 "nginx: worker process"
             ├─1308 "nginx: worker process"
             └─1309 "nginx: worker process"

Sep 15 10:38:58 ubuntu systemd[1]: Starting nginx.service - A high
Sep 15 10:38:58 ubuntu systemd[1]: Started nginx.service - A high
```

```
~ (1.054s)
systemctl status nginx

● nginx.service - A high performance web server and a reverse prox
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enable
     Active: active (running) since Tue 2026-09-15 10:38:58 -05; 3
```

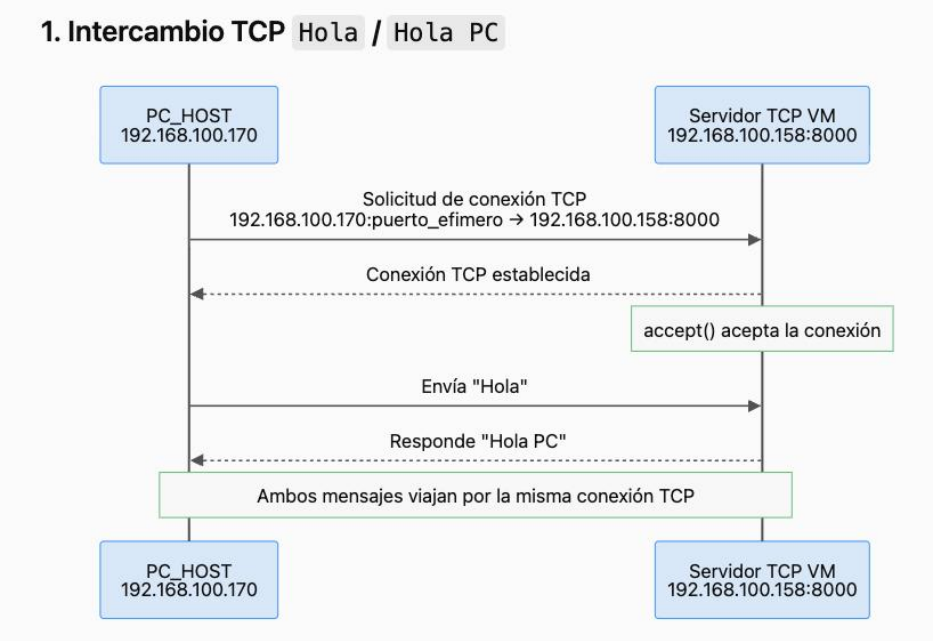
Parte 8 — Documentar la arquitectura implementada — 2 puntos

8.1. Elabora un diagrama de bloques propio de la arquitectura de red final, con los valores reales de tu entorno. Debe incluir:

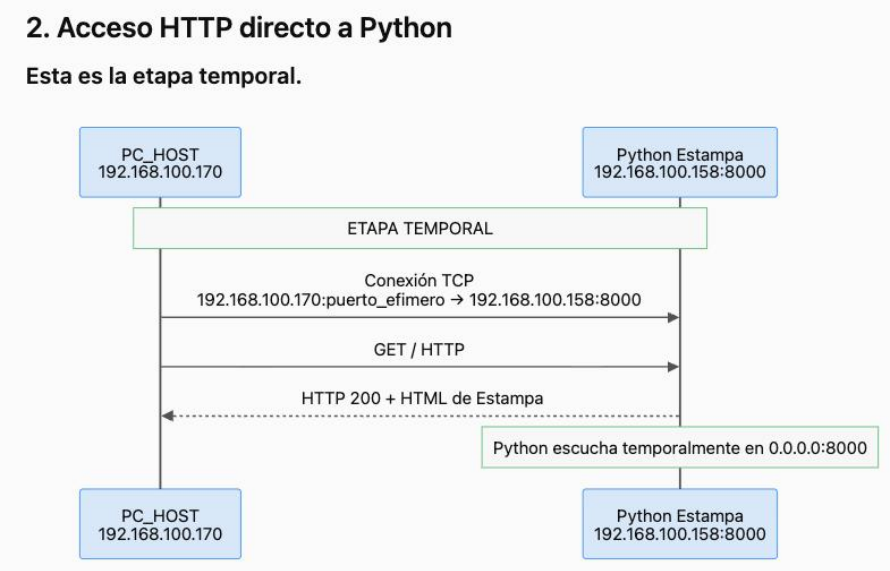
- **Equipo físico, sistema anfitrión, HOST_IP y cliente navegador o terminal.**
- **Hipervisor y su tipo, VM Ubuntu, recursos asignados, interfaz virtual, VM_IP y prefijo de subred.**
- **Adaptador puente, interfaz física, LAN y router con ROUTER_IP.**
- **UFW como control de acceso de la VM y las reglas relevantes.**
- **Nginx en VM_IP:80, Python en 127.0.0.1:8000 y archivos de ecommerce_webapp dentro de la VM.**
- **Flujo HTTP del cliente a Nginx y del proxy al backend, con flechas de respuesta, protocolos y puertos.**
- **Salida de la VM hacia Internet usada en la parte 2: resolución DNS, destino observado y NAT del router, diferenciada del acceso local a la landing.**
- **Representa correctamente la pertenencia de componentes: Nginx y Python son procesos dentro de la misma VM; 127.0.0.1 pertenece a esa VM. El host aporta la interfaz física al puente, pero su IP no reemplaza la IP de origen de la VM. No dibujes el router como salto obligatorio de la petición local si ambos equipos comparten subred.**

Además, entrega cuatro diagramas de secuencia propios:

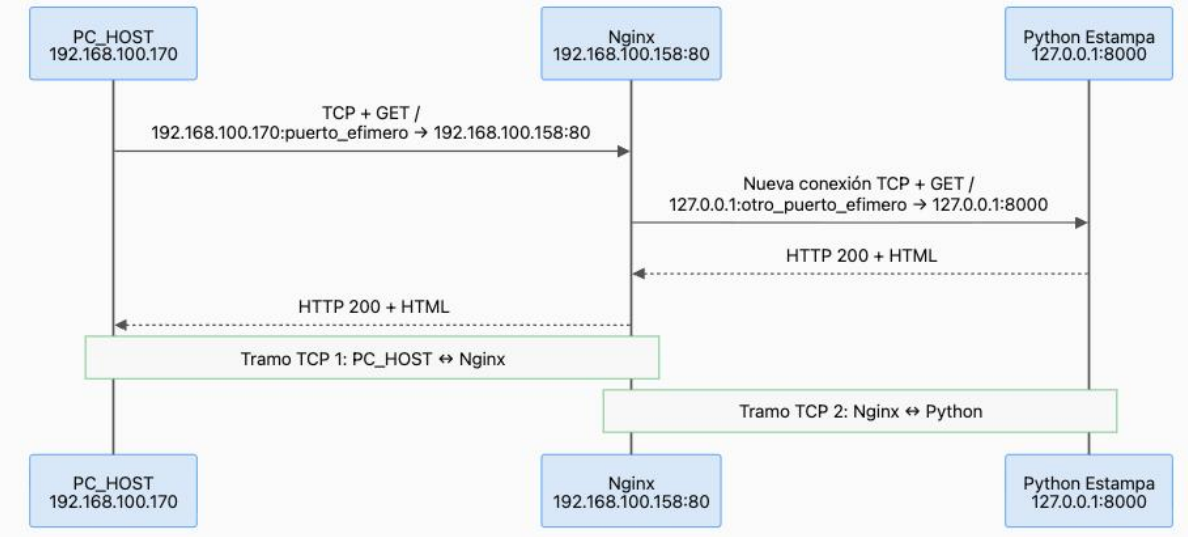
Intercambio TCP Hola / Hola PC, cubriendo los dos diagramas de saludo proporcionados.



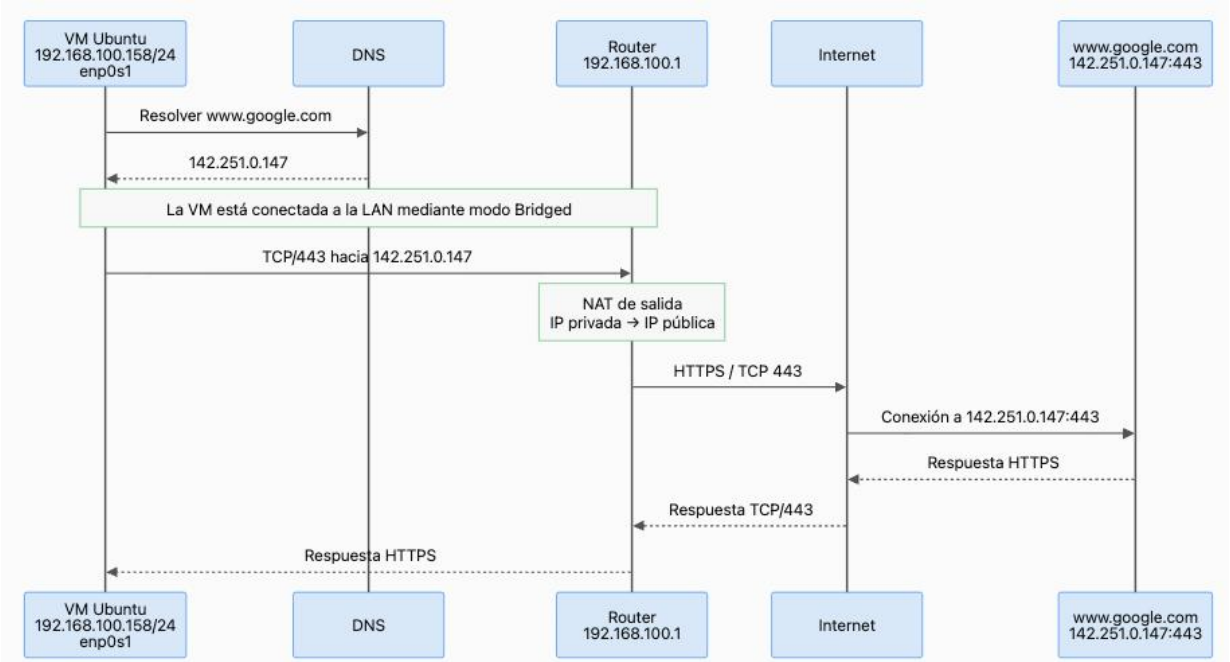
Acceso HTTP directo a Python durante la etapa temporal, usando GET /.



Acceso HTTP final mediante Nginx, con los dos tramos TCP y el retorno del HTML.

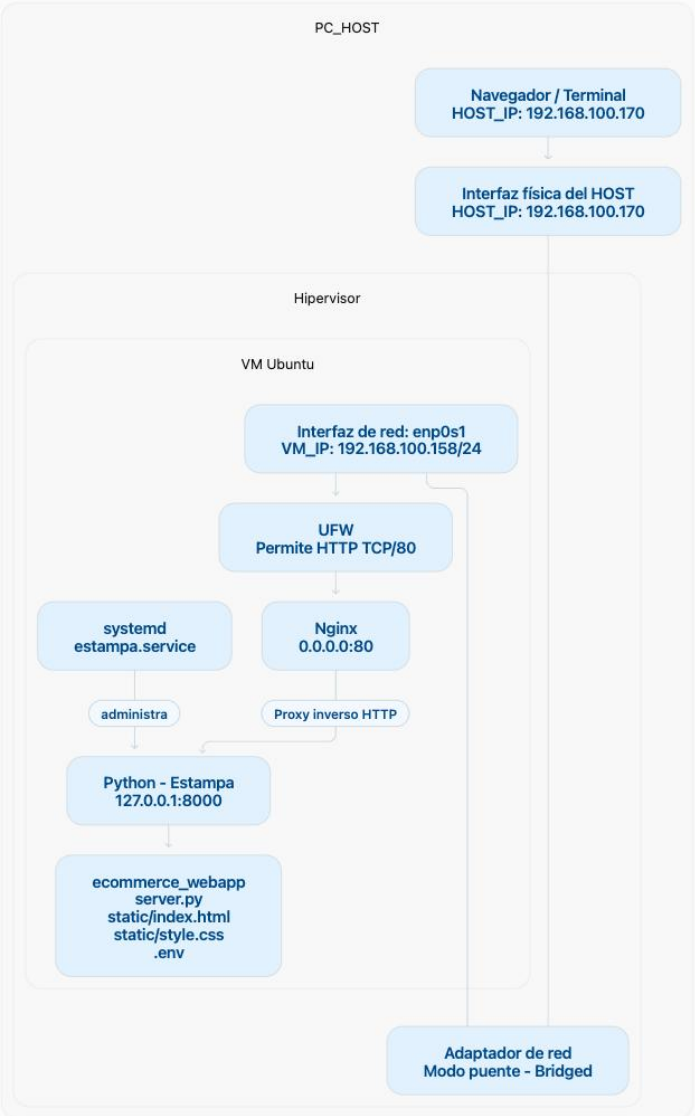


Salida HTTPS desde la VM hacia Internet, retomando tarea, con DNS, puente y NAT del router.



Puedes utilizar Mermaid, draw.io u otra herramienta. Entrega el archivo editable y su representación legible en el informe. Rotula el acceso directo a 8000 como etapa temporal para no confundirlo con la arquitectura final.

E19: diagrama de bloques y cuatro diagramas de secuencia con IP reales, puertos, protocolos y leyenda.



Actividad adicional obligatoria — Mapa conceptual en .canvas — 2 puntos

- **Elabora un mapa conceptual que relacione todos los conceptos que consideres relevantes desde la sesión 00 hasta la sesión 10. Selecciona los conceptos con criterio y explica sus conexiones con el despliegue realizado. Esta actividad forma parte de la calificación sobre 20 puntos; no otorga puntos por encima de ese total.**

- **Revisa las sesiones 00 a 10 y selecciona los conceptos relevantes de cada material disponible. Las sesiones 03 y 04 comparten un solo archivo de diapositivas.**

- **Organiza el mapa alrededor del despliegue y la administración de la landing ecommerce. Puedes agrupar los conceptos por temas, conservando una referencia a la sesión de origen de cada nodo o grupo.**

- **Conecta los conceptos con flechas etiquetadas que expresen una relación concreta, por ejemplo: «ejecuta», «escucha en», «reenvía a», «controla el acceso a» o «resuelve». Incluye conexiones entre sesiones, no solo listas independientes de términos.**

- **Considera, según su relevancia, sistemas operativos, nube, virtualización, hipervisores, recursos, Linux, terminal y shell, archivos, usuarios, permisos, servicios, systemd, interfaces, IP, subredes, puente, rutas, DNS, NAT, TCP, HTTP, puertos, Nginx, Python, proxy inverso, UFW y herramientas de asistencia. Esta lista orienta la selección; puedes incorporar otros conceptos fundamentados en las sesiones.**

- **Vincula los conceptos con ejemplos de tu implementación: qué proceso escucha en cada puerto, qué administra systemd, qué controla UFW y cómo llega una petición al backend. Distingue los conceptos introductorios del curso de las tecnologías que efectivamente implementaste.**

- **Guarda el mapa como PC1_Apellido_Nombre_mapa_conceptual.canvas, en formato editable compatible con Obsidian Canvas (JSON Canvas). Debe abrirse y mostrar sus nodos y conexiones; cambiar la extensión de una imagen o un PDF no cumple el requisito.**

- **Incluye en el informe una vista general legible y los acercamientos necesarios, junto con un párrafo que justifique tu selección y explique al menos tres relaciones entre contenidos de distintas sesiones.**

- **El mapa conceptual complementa el diagrama de bloques y los diagramas de secuencia: explica relaciones entre conocimientos, mientras los otros representan la arquitectura y sus comunicaciones. Debes entregar los tres tipos de representación.**

E20: mapa conceptual abierto en una herramienta compatible, con conceptos, conexiones etiquetadas y referencias a las sesiones visibles; adjunta también el archivo .canvas original.

